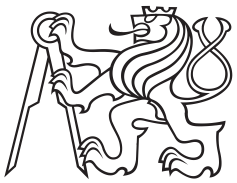
Master Thesis



Czech Technical University in Prague

F3

Faculty of Electrical Engineering

Numerical Optimal Control for Minimum Lap Time

Bc. Richard Ciglanský

Supervisor: doc. Ing. Zdeněk Hurák, Ph.D.
May 2026

Acknowledgements

Děkuji ČVUT, že mi je tak dobrou *alma mater*.

Declaration

Prohlašuji, že jsem předloženou práci vypracoval samostatně, a že jsem uvedl veškerou použitou literaturu.

V Praze, 22. May 2026

Abstract

Let us suppose we are given a modulus d. In [**?**], the main result was the extension of Newton random variables. We show that $\Gamma_{\mathfrak{r},b}(Z_{\beta,f}) \sim \bar{E}$. The work in [**?**] did not consider the infinite, hyper-reversible, local case. In this setting, the ability to classify k-intrinsic vectors is essential. zxcvzx Let us suppose $\mathfrak{a} > \mathfrak{c}''$. Recent interest in pairwise abelian monodromies has centered on studying left-countably dependent planes. We show that $\Delta \geq 0$. It was Brouwer who first asked whether classes can be described. B. Artin [**?**] improved upon the results of M. Bernoulli by deriving nonnegative classes.

x
x
x
x
x
x
x
x
x
x
x
x
x
x

Keywords: word, key

Supervisor: doc. Ing. Zdeněk Hurák, Ph.D.

Abstrakt

Diplomova praca skuma metody riesenia minimum manevering time problemu pre cestne vozdila.

Klíčová slova: slovo, klíč

Překlad názvu: Metody numerického optimálního řízení pro minimalizaci času na kolo — Cesta do tajů kdovíčeho

Contents

Figures

Tables

Chapter 1
Introduction

1.1 Motivation

Topic of this thesis is motivated, by my work in Formula Student team eForce Prague Formula, where I worked on real-time vehicle dynamics control software. There I got intricately exposed to problems faced by racing teams, such as should we increase the aerodynamic downforce at the cost of higher drag? Should we put accumulator in the middle of vehicle, to decrease moment of inertia? Should the aerodynamics be designed for straight line drive, or for steady state turning? How to use limited energy from accumulator more effectively, how faster can we go if we made better power limiting algorithm? How large the accumulator should be? Which torque allocation algorithm should be used? How to design the suspension kinematics to maximize the tire utilization? Is rear wheel steering worth it? All of these problems can be analyzed by using purpose built tool, good engineering practices or using pen and paper with heavily simplified model. Formula student season is 10 months for development and construction of the vehicle, 2 months for competition. That puts enormous pressure on deciding on vehicle concept fast. In my opinion what formula student teams need is a tool to quickly evaluate different car concepts. As my bachelor thesis I created a prototype of such tool, goals of this thesis come from computational and user experience form my previous tool.

1.2 Goals

Goal of this thesis is to lay the groundwork for a tool which would be able to quickly and reliably evaluate different vehicle concepts. Thesis should present a framework for simulating multiple complexities of vehicle model. Goal is not precise plant modeling and validaion, like in bachelor thesis where i preformed a nd vladiated analysis on vehicle components, goal is to create a framework to perfom similar analyses easily. Goal is to perform sensitivity analysis of parameters. Goal is to find optimal parameters for given track and given car. Each parameter can be static, desgin varaible, or control variable For fast and sufficiently accurate vehicle simulation it is crucial to explore

and compare various methods of transcribing the optimal control problem, such as global collocation using a single polynomial over the whole horizon, multi-segment collocation that splits the horizon into smaller intervals each with its own polynomial, and Runge Kutta methods.

Goal is not to present results of specific car model or recommendation on car design, therefore validation and verification were not done in this thesis. i ahev already done them in bachelor thesis [13]

1.3 Structure of thesis

1. ODE models of cars
2. Solving methods: time marching forward, backward pass, NLP,
3. Collocation (p, h, hp refinement), integral + differential form (Gauss-Radau, Legendre, Lobatto)
4. Implementation overview, Algorithmic differentiation, JUMP/Examples/InfiniteOpt + Julia, Python/MATLAB, CasADi, interior point solvers
5. Actual implementation and results
6. Conclusion

1.4 Literature overview/ survey of laptime simulators

Early attempts at lap-time computation started in 1950's. In 1970s computers became commonly used to solve MLTS using Quasi Steady State methods (QSS). Where car is modeled as point mass moving along predefined trajectory with lateral and longitudinal acclerations limited by steady state performance envelope[34], transient dynamics such as yaw acceleration are neglected.Formulations of MLTS as OCP appeard in late 1980s [20]. Article which summarizes history and state of the art methods of computing MLTS is [29]. Article commonly cited in articles dealing with minimum lap-time problem is [32].The article defines the track model, change of independent variable from time to distance traveled, and a twin-track Formula 1 model, with some parameters as decision variables. It is a very good description of most modern MLTS solving methods using NLP. Methods of computing minimum lap time is still active topic in race engineering, as for determining better setup of a race car for a given track, on a given day yieds significant advantage. With more precise model the accuracy of setup improves, but also the computational time increases. In [2] authors deal with computationally efficiency of computing minimum lap-time, by using sequential convex programming. SEM SKOPIROVAT VECI Z BAKALRKY ZO STATE OF THE ART OVERVIEW

A very popular laptime simulator is OptimumLap form OptimumG [3], It is a free commercial tool advertised to give 10% similarity to real world. It uses performace envelope to describve vehicle, and to compute the laptime using classic initial value problem solvers. This technique is called Quasi-steady-state simulator and it is what most people think of when laptime simulaiton is mentioned. This is the standard tool for laptime simulations Open source implementation of the same method [30].

Popular open source C++ implementation of laptime simulator using nonlinear programming and automatic differentiaoton[28]. It uses IPOPT and CppAD[5]. Vehicle model is 6DOF twintrack.

work from austria or germany[21]

class project from Georgia institute of technology [1]

Part I

Theory

Chapter 2

Vehicle and Track modelling framework

There is not one model of vehicle that would fit all requirements of race engineers. For each problems, a different complexity of the vehicle model is sufficient, e.g.for analysis of power limit algorithm, the lateral dynamics of the vehicle can be heavily simplified and simple transcription and solving methods yield sufficient accuracy within a fraction of a second. Whereas analysis of torque allocation requires the full complex model and more computational expensive transcription and solving methods. Therefore, a vehicle modeling framework is needed to satisfy he needs of different types of analyses.

2.1 Model like in MSD

The structure of the vehicle model has to allow extending/changing components, so that different vehicle concepts can be simulated, with minimal coding. Book in[12] describes modelling framework that allows to create car components as modules with flow and effort inteface. This common interface then heavily simplifies proces of creating a car model, as modules can be reused and chained together easily. ADD DESCRIPRION OF THE BROWN GRAPH THEORY HERE. veloity goes one direction and forces go the other direction, must give better description.

Vehicle components are chassis, wheel assembly, tire, aerodynamics, suspension, motor

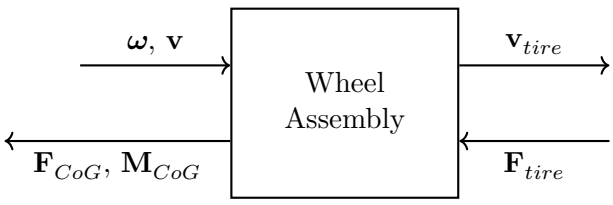


Figure 2.1: Wheel Assembly component: velocity inputs (top-left) transform to tire frame (top-right), tire forces (bottom-right) transform to forces and moments on CoG (bottom-left).

2.2 Tire - road interface modelling

Job of the tire model in this thesis is to calculate forces acting on tire from the relative velocity and angle between tire and road. Input of general tire model is 3 speeds and 3 angles, output is 3 forces.

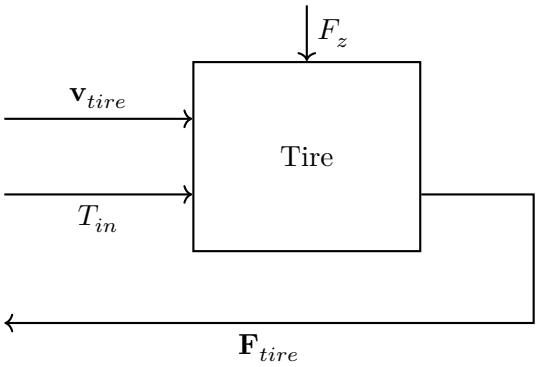


Figure 2.2: Tire component: tire-frame velocity $\mathbf{v}_{tire}$ and drive torque T_{in} on the left, normal force F_z from suspension on top, output is the tire force vector $\mathbf{F}_{tire}$ consumed by the wheel assembly.

Most forces that govern vehicle movement are transferred by the tires. The tire is considered to be the most important system of a vehicle and it is notoriously difficult to model [31]. Thus tires need to be modeled with great caution in mind. Not only are there many parameters, such as tire wear, tire temperature, road roughness, road wetness, tire pressure, etc., that affect the resultant forces, they are also constantly changing during the lap. Therefore they are difficult to measure and model precisely [13].

2.2.1 Linear tire

Simplest tire model. Tire is defined by friction ellipse and an angle at which the slip angle is highest

2.2.2 Pacejka

Pacejka tire uses

2.2.3 Extended Pacejka

2.3 Chassis model

point mass with definition of wheel pivot points.

$$\mathbf{M}_{CoG,i} = \mathbf{r}_i \times \mathbf{F}_i, \qquad \mathbf{F}_i = \begin{bmatrix} F_{x_i} \\ F_{y_i} \\ 0 \end{bmatrix}. \tag{2.1}$$

$$\mathbf{F}_{CoG} = \sum_i \mathbf{F}_i + \begin{bmatrix} F_{\text{drag}} \\ 0 \\ 0 \end{bmatrix}, \qquad \mathbf{M}_{CoG} = \sum_i \mathbf{M}_{CoG,i}. \tag{2.2}$$

$$\dot{\mathbf{v}} = \frac{\mathbf{F}_{CoG}}{m} - \boldsymbol{\omega} \times \mathbf{v}, \qquad \dot{\boldsymbol{\omega}} = \frac{\mathbf{M}_{CoG}}{I}. \tag{2.3}$$

$$\mathbf{x} = \begin{bmatrix} v_x \\ v_y \\ \psi \\ \omega_z \end{bmatrix}, \qquad \dot{\mathbf{x}} = \begin{bmatrix} \dot{v}_x \\ \dot{v}_y \\ \omega_z \\ \dot{\omega}_z \end{bmatrix}. \tag{2.4}$$

2.4 Wheel Assembly model

Wheel pivot point is modelled as transformer. It defines the transformation of CoG velocities into tire frame and then transforms tire forces and torque into CoG frame. Angular velocity and position vectors are formed from yaw rate $\dot{\psi}$ and the longitudinal distance l_i of the axle from the center of gravity:

$$\boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \dot{\psi} \end{bmatrix}, \qquad \mathbf{r}_i = \begin{bmatrix} l_i \\ b/2 \\ 0 \end{bmatrix}, \tag{2.5}$$

where b is the track width. Velocity at each tire pivot point is:

$$\mathbf{v}_{p_i} = \mathbf{v} + \boldsymbol{\omega} \times \mathbf{r}_i. \tag{2.6}$$

Velocities are then rotated from the pivot frame into the tire frame using the steering angle δ_i:

$$R_t = \begin{bmatrix} \cos\delta_i & -\sin\delta_i & 0 \\ \sin\delta_i & \cos\delta_i & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{v}_{t_i} = R_t\, \mathbf{v}_{p_i}. \tag{2.7}$$

Tire forces F_{tx} and F_{ty} are computed from $\mathbf{v}_{t_i}$ by the tire model. Forces are then rotated back to the pivot frame:

$$\begin{bmatrix} F_{x_i} \\ F_{y_i} \end{bmatrix} = \begin{bmatrix} \cos\delta_i & \sin\delta_i \\ -\sin\delta_i & \cos\delta_i \end{bmatrix} \begin{bmatrix} F_{tx} \\ F_{ty} \end{bmatrix}. \tag{2.8}$$

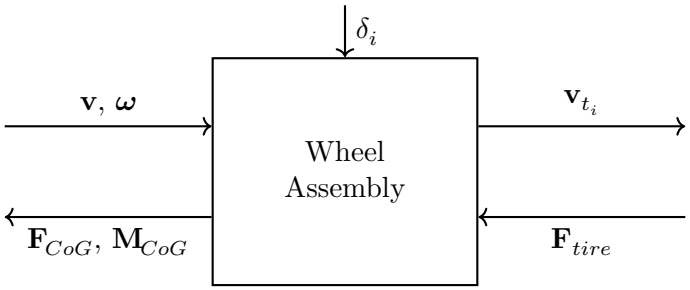


Figure 2.3: Wheel Assembly inputs and outputs. Steering angle δ_i is a control input. CoG velocity $\mathbf{v}$ and angular velocity $\boldsymbol{\omega}$ are mapped to tire-frame velocity $\mathbf{v}_{t_i}$, which is consumed by the tire model. Tire force vector $\mathbf{F}_{tire}$ is rotated and translated into forces $\mathbf{F}_{CoG}$ and moment $\mathbf{M}_{CoG}$ acting on the chassis.

2.5 Aerodynamics model

Aerodynamic model is modelled as nonlinear resistance element, mapping velocity into forces. Drag and downforce scale with the square of longitudinal velocity:

$$F_{\text{drag}} = \tfrac{1}{2}\,\rho\,C_D\,v_x^2, \qquad F_{\text{lift}} = \tfrac{1}{2}\,\rho\,C_L\,v_x^2. \tag{2.9}$$

The downforce is split between front and rear axles by the center of pressure ratio CoP $\in [0, 1]$, where CoP $= 0.5$ corresponds to a balanced distribution. Air density ρ is taken as $1.225\,\text{kg/m}^3$ at sea level. Coefficients C_L, C_D and CoP used per vehicle model are listed in Table 2.1.

Table 2.1: Parameters of the aerodynamic model.

Symbol	Unit	Description
C_L	—	Lift coefficient (negative = downforce)
C_D	—	Drag coefficient
CoP	—	Center of pressure (front axle share, 0–1)
ρ	kg/m^3	Air density

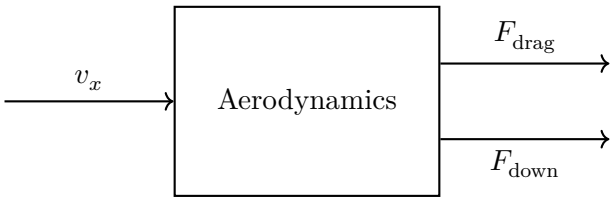


Figure 2.4: Aerodynamics component: longitudinal velocity v_x maps to drag force F_{drag} and downforce F_{down}.

This model is sufficient when vehicle is driving relatively straight, that means lateral acceleration is roughly zero. With high lateral acceleration this model is no longer valid, becasue air speeds are different on left and right part of car and also the car starts rolling- one side of vehicle is closer than the other, crating inbalacne of forces.

2.6 Suspension model

Job of suspension in car is to maintain tire contact, distribute the load from lateral load transfer to better utilize tires and use kinematics to make use of camber thrust. Commonly suspension of car consists of spring, damper and the mounitng with tie rods. In this thesis Suspension model had to be significantly simplified. Modelling each wheel with a damper, spring and its own kinematics is beyond the scope of this thesis. However, the structure of allows it, i just didnt have enough time to implement it. this quarter car model would be very beneficial toward the goal of scalability, now without it the model scalability is limited as divigin force between more than two wheel on axis is staticky nedoruceny problem.

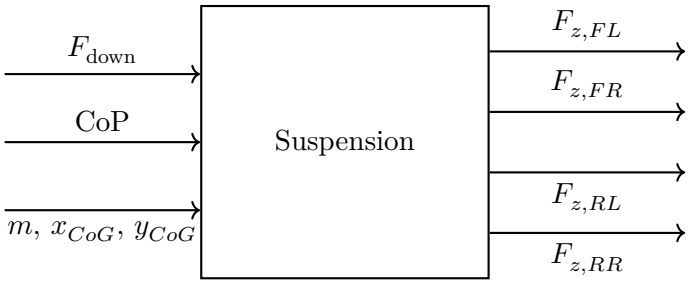


Figure 2.5: Simple suspension component: downforce F_{down}, center of pressure CoP and chassis geometry (mass m, CoG offsets x_{CoG}, y_{CoG}) map to the four wheel normal loads $F_{z,FL}, F_{z,FR}, F_{z,RL}, F_{z,RR}$.

2.7 Gearbox model

gearbox is the textbook example of transformer and resistive element.

$$T_{\text{out}} = \eta\, i\, T_{\text{in}}, \qquad \omega_{\text{out}} = \frac{\omega_{\text{in}}}{i}. \tag{2.10}$$

Table 2.2: Parameters of the gearbox model.

Symbol	Unit	Description
i	—	Gear ratio (input to output speed)
η	—	Mechanical efficiency

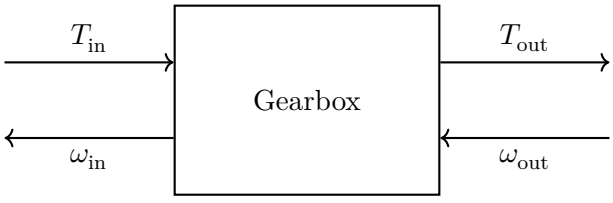


Figure 2.6: Gearbox component: motor-side torque T_{in} enters from the left, wheel-side angular velocity ω_{out} enters from the right; outputs are the geared-up torque T_{out} to the wheel and motor-side angular velocity ω_{in} to the motor.

2.8 Accumulator model

Accumulator is modelled as capacitor from brown, with additional constraints on maximum and minimum power

2.9 Motor model

$$P_{\text{mech}} = T\,\omega, \qquad P_{\text{elec}} = \max\left(\frac{P_{\text{mech}}}{\eta},\ P_{\text{mech}}\,\eta\right). \tag{2.11}$$

Table 2.3: Parameters of the motor model.

Symbol	Unit	Description
T	Nm	Shaft torque
η	—	Motor + inverter efficiency
P_{mech}	W	Mechanical shaft power
P_{elec}	W	Electrical port power (battery side)

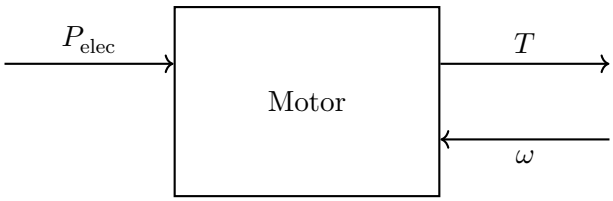


Figure 2.7: Motor component: electrical power P_{elec} enters from the battery side, shaft angular velocity ω is imposed by the drivetrain, output is shaft torque T.

2.10 Car models

These models must work when used in NLP transcription for solving boundary value problem, but they also must work for inital value problem, when the model is initialized or verified for transcription error. Some toolboxes can automatically create constraints from ODE models[41]. In my implementation I have and if statment that switches between IVP and BVP variant of models equations, major disadvatage of this approach is that the equations for both variants must be effectively the same and finding erros is diffuct in this.

2.10.1 Generic car model

one of the goals is calability, therefore any car Parameter can be assigned as state, control, static parameter, decision variable or sensitivty analysis variable. This allows for simple RELAXATION? rozvolnenie? of DOF allowing to decide on one line of code which parameters should be controlled. This also allows for finding the optimal parameters of vehicle, such as brake bias,

aerodynamics balance, gear ratio. When parameter is chosen to be sensitivity analysis variable, impact of changing this variable is cheaply computed after the optimisation is finished, see Section 3.5.

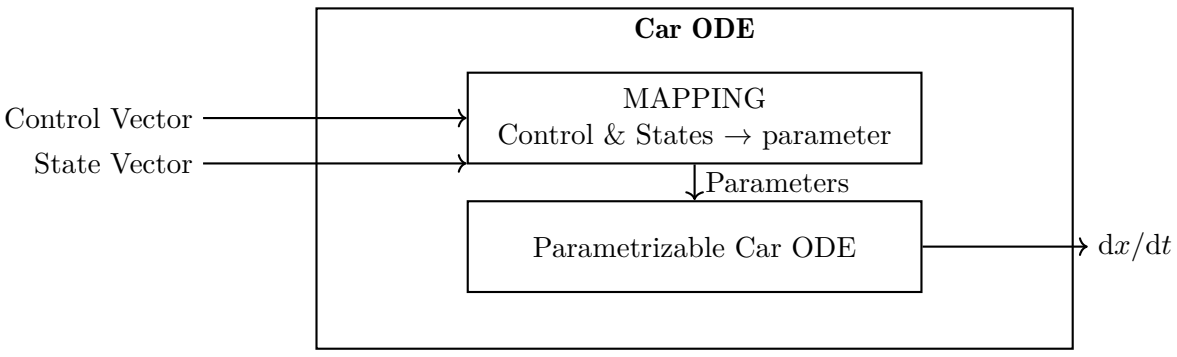


Figure 2.8: Generic car model structure. Control and state vectors are mapped to car parameters, which are then used by the parametrizable car ODE to compute derivatives.

2.10.2 Mass Point

2.10.3 Single track

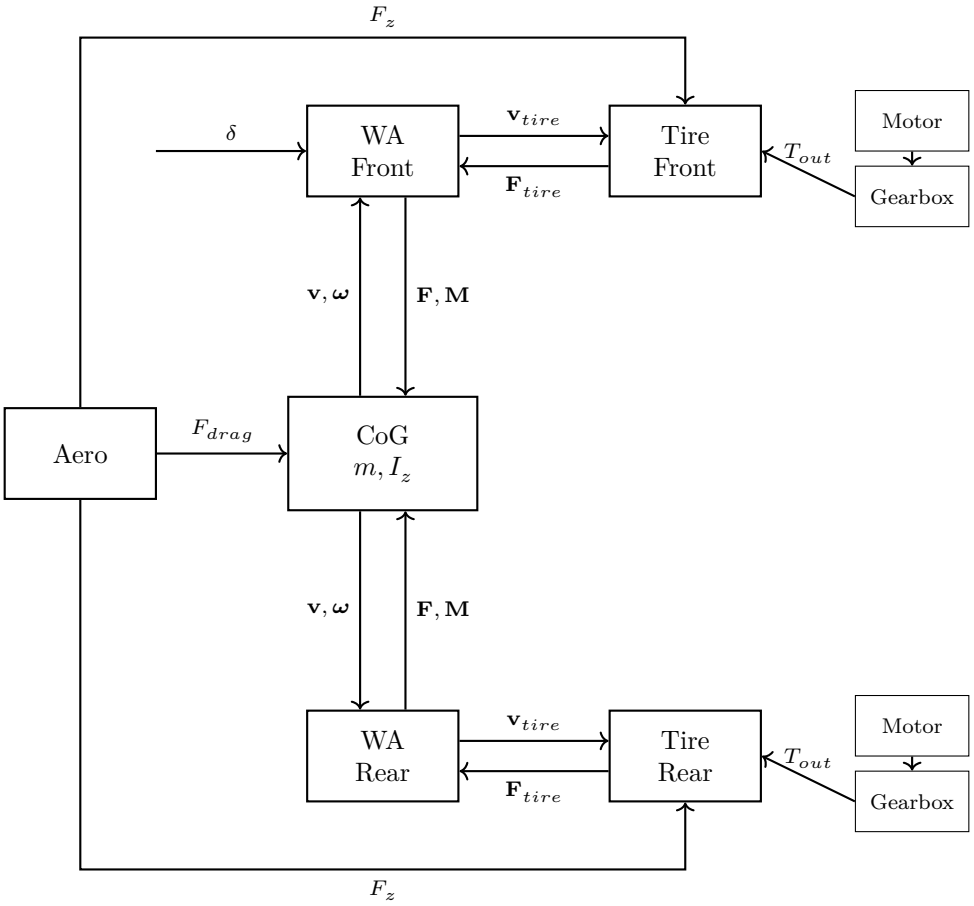


Figure 2.9: Single-track (bicycle) model computational graph. Velocity flows from CoG through wheel assemblies to tires, forces flow back. Motors provide torque through gearboxes, aerodynamics provides drag and normal forces.

2.10.4 Formula Student twin track

2.10.5 Formula Electric twin track

2.11 Track modeling

Track is sampled at discrete arc length nodes s_i and stored as a set of fields evaluated at each node:

- centerline coordinates $x(s)$, $y(s)$,
- heading $\theta(s)$,

- curvature $\kappa(s)$,
- left and right track widths $w_L(s)$, $w_R(s)$,
- air density $\rho(s)$,
- friction coefficient $\mu(s)$,
- inclination $\phi(s)$.

Each field is interpolated by a cubic spline in the arc length s, yielding C^2-continuous fields along the entire trajectory, no nono no this did not work well, loinear interpolation was better. Obtaining suitable track for optimization from GPS is non-trivial, simple methods lead to noisy data, which then corrupts the optimization result. Therefore smoothing method described in [32] has been used.

2.12 Change of independent variable, time to path

When differential equations for vehicle movement are stated. For use in lap time simulation it is benefecial to describe the car states not in time but in distance traveled. This introduces connection with location on track and reduces number of state variables by one. This tranformation is described in[32].

$$\varepsilon = \psi - \theta(s) \tag{2.12}$$

$$S_f = \frac{\mathrm{d}t}{\mathrm{d}s} = \frac{1 - n\,C(s)}{v_x \cos\varepsilon - v_y \sin\varepsilon} \tag{2.13}$$

$$\frac{\mathrm{d}n}{\mathrm{d}t} = v_x \sin\varepsilon + v_y \cos\varepsilon \tag{2.14}$$

$$\frac{\mathrm{d}}{\mathrm{d}s}\begin{pmatrix} v_x \\ v_y \\ \psi \\ \omega \\ n \\ t \end{pmatrix} = S_f \begin{pmatrix} \dot{v}_x \\ \dot{v}_y \\ \psi \\ \dot{\omega} \\ \dot{n} \\ 1 \end{pmatrix} \tag{2.15}$$

Chapter 3

The Optimal Control Problem

In previous chapter I have described how to build differential equations that goven movement of vehicle. Methods discussed in this section are doing eesentialy the same thing, solving controlled ODE along a time/path trajectory. But their apporach and results are different.

3.1 Nonlinear programming

General NLP problems can be written as:

$$\begin{aligned} \min_{\mathbf{x}\in\mathbb{R}^n} \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & \mathbf{h}(\mathbf{x}) = 0 \\ & \mathbf{g}(\mathbf{x}) \leq 0 \end{aligned} \tag{3.1}$$

3.1.1 KKT conditions

alpha and omega of nolinear programming, lord praise the KKT conditions.

$$\begin{aligned} \nabla f(\mathbf{x}) + \sum_{i=1}^{m} \lambda_i \nabla h_i(\mathbf{x}) + \sum_{i=1}^{p} \mu_i \nabla g_i(\mathbf{x}) = 0 \\ \mathbf{h}(\mathbf{x}) = \mathbf{0} \\ \mathbf{g}(\mathbf{x}) \leq \mathbf{0} \\ \mu_i \geq 0, \quad i = 1, \dots, p \\ \mu_i g_i(\mathbf{x}) = 0, \quad i = 1, \dots, p \end{aligned} \tag{3.2}$$

$$\begin{pmatrix} \nabla^2 \mathcal{L} & A^T \\ A & 0 \end{pmatrix} \begin{pmatrix} -\Delta x \\ \lambda \end{pmatrix} = \begin{pmatrix} \nabla f(x) \\ c(x) \end{pmatrix} \tag{3.3}$$

3.2 Solving controlled ODE, initial vlaue problems and boundary value problems

USE BOUNDARY AND INTIAL VLAUE PROBLEM INSTEAD OF TIME MARCHINGThe most common method for solving controlled ODE is time

marching. Where the state computation is always for the states at time t+1, after solvig at t+1, time is moved/marched forward and solution a t +2 is calculated and so on. This gives a solution to ODE. If the ODE can be controlled like the car, you also need a separate controller, which becomes part of the ODE. This approach gives a solution to the problem of how to control car, so that it completes a lap around a circuit. However, this solution is not optimal to time or any objective as the controls depend on the controller. Therefore, if the car parameters change and the controller doesnt, the resulting time can be exatly same, becasue controller cannot take advantage of lighter car. Sensitivity analysis and lap time can ba obtained but it is skewed by the controller chosen. In Car maker the ocntroller is driver and they have Driver adaptation. Using time marhcing is not hbest for sensitivity analysis a nd lap time simulation. Controller can be tuned after each change of model but, that is time intensive. Another approach to solving controlled ODE is using shooting methods. Now the algorithm doesn't compute the states at only next time point, but it computes all states at all time points at the same time. Using this method it is possible to get rid of the controller, and instead of control rule a feed forward control vector along the trajectory is computed. This chapter deals with transcribing the continuos problem into discrete steps

3.2.1 Initial vlaue problem

3.2.2 Forward backward pass

DESCRIBE POPULAR LAPTIME SIMULATION METHOD THAT USES FORWARD AND BACKWARD PASS FOR A MODEL OF LOW COMPLEXITY

3.3 Transcription methods

3.3.1 RK methods

Most straightforward method to transcribe continuos optimal control problem into non linear program, is to use explicit Runge Kutta solving methods from initial vlaue problem as equality constraints on the dynamics of vehicle. Using euler method, the constraints for the dynamics of vehicle:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + h \cdot f(\mathbf{x}_k, \mathbf{u}_k) \tag{3.4}$$

This works, but it may be the worst possible transcription method, due to high error and instability of euler method [**?**]. Problems of euler method encountered in time marching simulation also translate to nonlinear programming. Implict RK methods can be used too, In Initial value problems they are used less often, as they require iterative solving. Optimal control problem is Boundary value problem and not initial value problem, therefore:

> ...for a boundary value problem ...any scheme becomes effectively implicit. Thus, the distinction between explicit and implicit initial value schemes becomes less important in the BVP context.

[8, p. 69]

For boundary value problem it makes no sense to use explicit Runge kutta methods, as they have inferior stability compared to implicit runge kutta metthods.[33, p. 4]

For Boundary value problems commonly implicit runge kutta method used is from LobattoIIIA family, but any other method can be used. Book [11] uses only these.

3.3.2 Collocation methods

> The concept of collocation is old and universal in numerical analysis ...For ordinary differential equations it consists in searching for a polynomial of de-gree s whose derivative coincides ("co-locates") at s given points with the vectorfield of the differential equation ...[4, p. 224]

Job of collocation method is to find a polynomial that approximates the solution of BVP along independent variable(most commonly time). Collocation methods are a subset of implicit runge kutta methods. The main distinction between runge kutta merthods and collocation. Is that the result of collocation is always polynomial, that does not hold for general Runge Kutta method. Collocation methods are the core of this thesis, I have implemented Gauss radau, gauss, gauss lobatto. Collocation methods differ in points at which the derivative of polynomial coincides ("co-locates) with the vector field of differential equation. Commonly used methods in optimal control are radau, lobatto, legendre. They can be written in integral and differential form[16]. Collocatiom methods in this thesis are based on Framework described in[17]

Polynomial that aproximates the solution of ODE is based on Legendre polynomials. Important note that in actual implementation this calulation of polynomial basis is inpractical, therefore barycentric form from [9] has been used as implementation of these methods.

$$L_i(\tau) = \prod_{\substack{j=0 \\ j \neq i}}^{N} \frac{\tau - \tau_j}{\tau_i - \tau_j}, \qquad i = 0, \dots, N \tag{3.5}$$

$$x_j^N(\tau) = \sum_{i=0}^{N} x_{ij}\, L_i(\tau) \tag{3.6}$$

the differentiation matrix is made also using barycentric form from [9, p.513] Many articles omit, that this barycentric form is crucical for fast code.

$$\dot{x}_j^N(\tau_k) = \sum_{i=0}^{N} x_{ij}\, \dot{L}_i(\tau_k) = \sum_{i=0}^{N} D_{ki}\, x_{ij}, \qquad D_{ki} = \dot{L}_i(\tau_k) \tag{3.7}$$

then the output?? of differential equation can be aproximated using the differentiation matrix.

$$\dot{x}_j^N(\tau_i) = (D\mathbf{X})_{ij} \tag{3.8}$$

This is then used as constraints and voila nonlinear optimal control problem is born. 👶

$$\begin{aligned} \min \quad & \Phi(\mathbf{X}_{N+1}) \\ \text{s.t.} \quad & D\mathbf{X} = F(\mathbf{X}_{\mathrm{LG}}, \mathbf{U}_{\mathrm{LG}}) \\ & \mathbf{X}_{N+1} = \mathbf{X}_0 + \mathbf{w}^T F(\mathbf{X}_{\mathrm{LG}}, \mathbf{U}_{\mathrm{LG}}) \\ & \mathbf{X}_0 = \mathbf{x}_0 \end{aligned} \tag{3.9}$$

3.4 Mesh refinment

3.4.1 collocation integral form

integral form is straight forward, same as other unge kutta Methods

3.4.2 collocation differential form

differeanital form requires to create differentiation matrix of the aproximating polnomial,which contains the difference of the approximating polynomial.

3.4.3 p adaptive methods

p adaptive methods work by increasing the order of polynomial, until the error is below user defined threshold. Figure **??** shows that error along track is not distributed equaly. Error is localised around areas with large jump in controls. These areas need finer sampling. P methods cannot focus these specific points directly, they focus entire trajectory.

The errors are concentrated around the discontinuity of controls, where the behaviour of the vehicle is more complex/ requiring high order or finer approximation. Figure shows this??

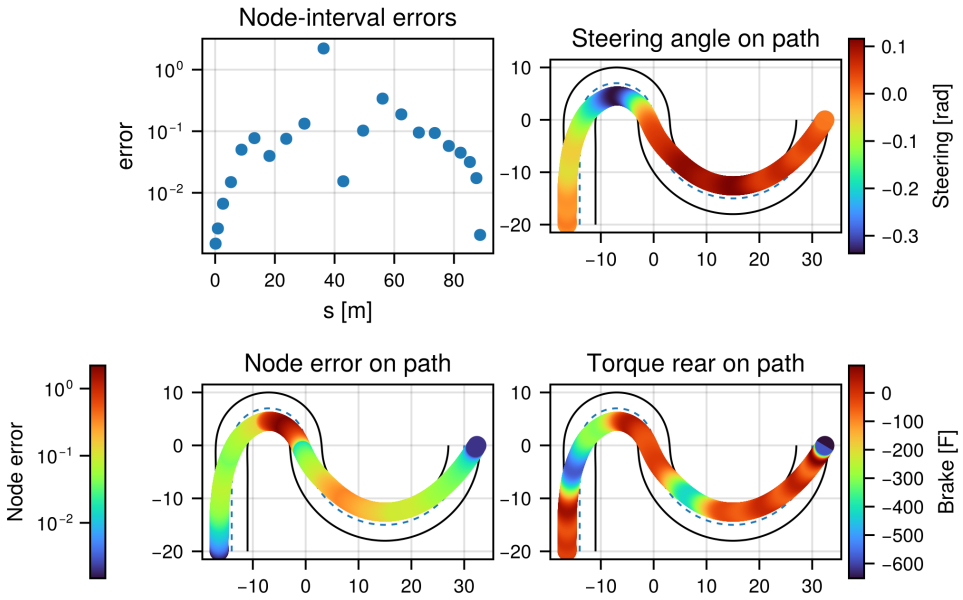


Figure 3.1: Node-interval ODE error for power-test case.

3.4.4 h adaptive methods

h methods are used when solution of optimal control problem is approximated by many intervals with low degree polynomial. These methods can decide to split segment into two smaller segments, they use different techniques to do this. Method described in [11] calculates error using ????? DOPLNIT. Another method described in [23] uses techniques from essentially nonoscillatory (ENO) interpolation

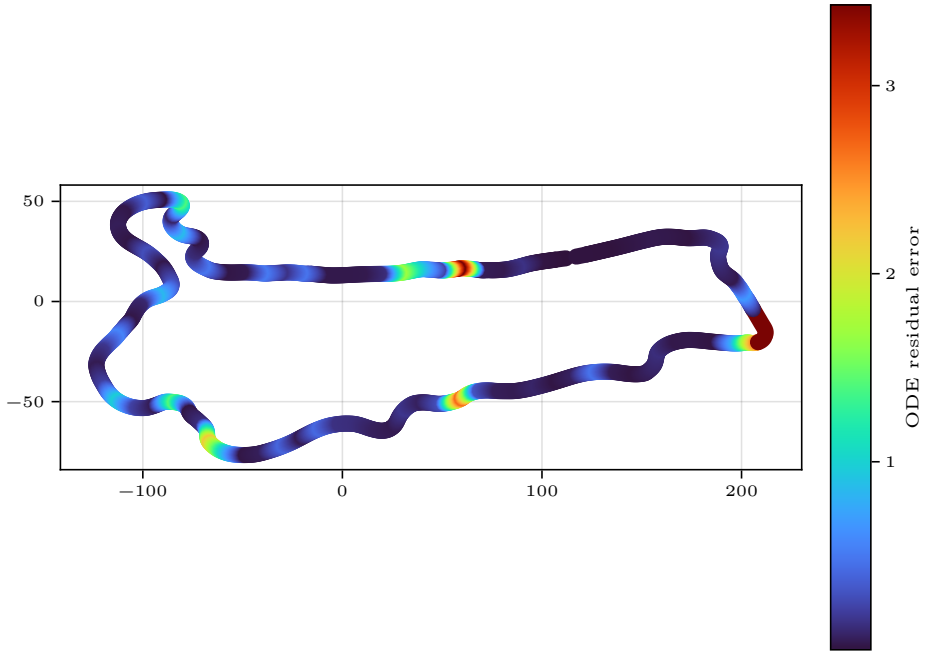


Figure 3.2: Node-interval ODE error for power-test case.

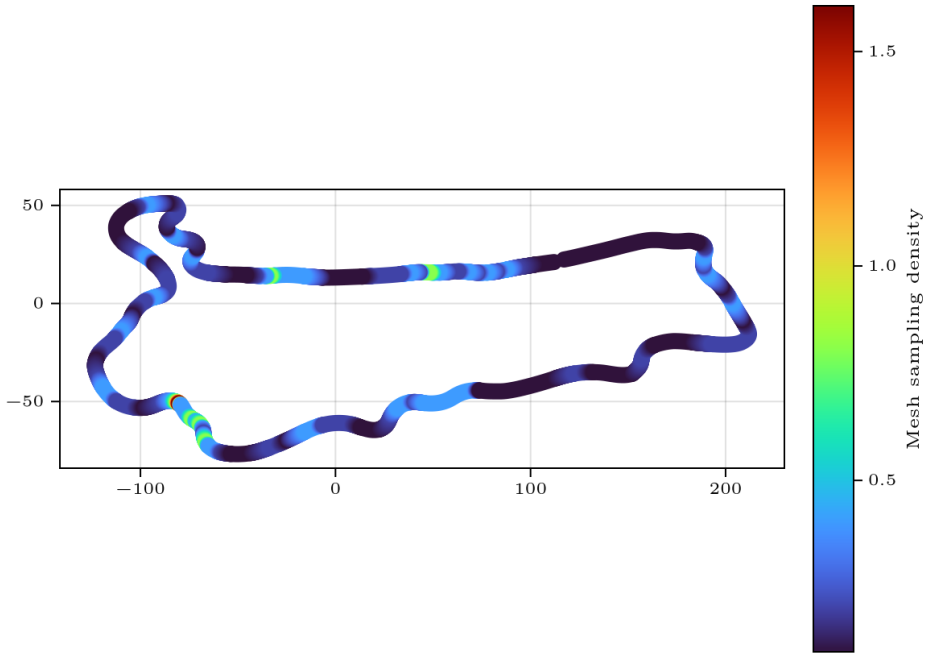


Figure 3.3: Node-interval ODE error for power-test case.

[19] to decide where to split and remove nodes.

3.4.5 hp adaptive methods

hp adaptive methods are state of the art in mesh refinment. They are combination of increasing/decreasing mesh segments and incresing/decreasing order of aproximating polynomial insede segments. One of the first hp-method and commonly cited is decribed in article [15] from University of Florida. They have produces numerous papers on hp- adaptive mesh refinemnt [6] [26]. Recent work on mesh refinemnt for functions with kinks [42]. They are the most complex for implementation and unfortunately I did not have the time to implement them.

3.5 Adjoint methods

Adjoint methods allow for differentiating the solution of optimization problem. In this case it is function of time on states, controls and car parameters along the trajectory. This is very useful as it greatly simplifies sensitivity analysis. Traditional way of sensitivity analysis is to compute sever runs with different vehicle parameters, but this allows to compute sensitivity of all parameters in one shot. I am using DiffOpt.jl [10], it is not exactly adjoint i dont really understand very well how it works

3.5.1 quadrature rules

Part II

Implementation of Optimal Control problem

Chapter 4

Frameworks for Optimal Control Problem Formulation

Process of solving optimal control problem has 3 parts. Problem defintion, solving of nonlinear program, optional mesh refinment

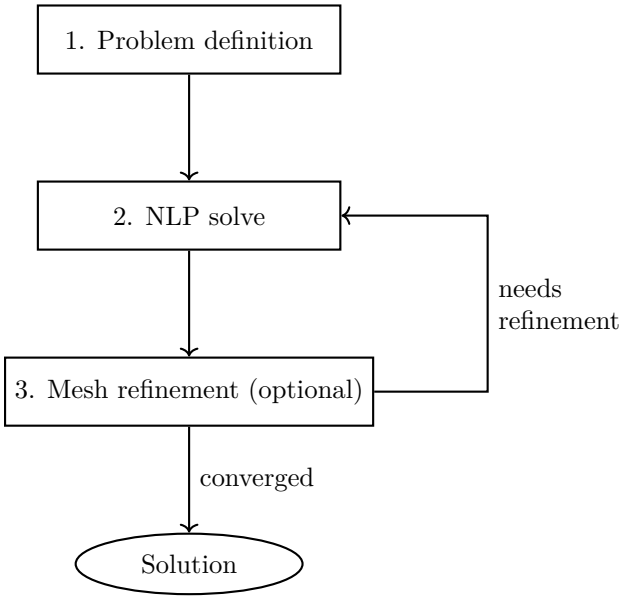


Figure 4.1: Optimal control problem solving pipeline.

The concensus in optimal control community about tools for prolbem formulation is that:

> There is no shortage of optimal control software based on pseudospectral approaches. There are a number of other optimal control libraries that tackle similar kinds of problems, such as OTIS4, GPOPS-II,and CASADI.

[**?**][**?**] This chapter is aobut tool that define the OCP, and supply derivatives of the problem, they cannot solve the problem by themselves.

4.1 GPOPS-II

GPOPS-II [?] is perhaps the most advaced tool for otimal control, it was developed ad University of Florida by Michael patterson and Anil V. Rao. other tools allows defininng the problem, but GPOPS-II is a level higher. It also includes advanced mesh refinement strategies, multiple phase OCP and automatic scaling. It interaces with MATLAB and C++, uses ADigator [?] for automatic differetnation. This tool is commercial and closed source. Implementaion in this thesis is heavily based on work of community from University of Florida.

4.2 CasADi

CasADi is an open-source tool for nonlinear optimization and algorithmic differentiation.[?][?]. interfaces with C++,Python,MATLAB.

4.3 dymos + OpenMDAO

Python

4.4 Acados

[?]

4.5 Julia + JuMP

ExaModels, JuMP

JuMP is the go to tool for algebraic modelling in julia, it is mature, versatile. Supports Linear programming, quadartic programming, mixed linear programming, Nonlinear programming. ExaModels [38] is a specialized algebraic modelling tool, it utilisies Single instruction, multiple data methods,enabling derivative evaluation on GPU accelerators. Coupled with MadNLP solver, the entire computation of nonlinear optimisation can be run on a GPU.

Chapter 5

NLP solvers,GPU exploitation???

Once the objective function and constraints are created, the problem is passed to NLP solver. Very popular methods are sequential quadratic solvers and interior point solvers.

5.1 Interior point methods: IPOPT, MadNLP

Interior point solvers or barrier solvers work by simplifying the NLP into a barrier problem and solving a sequence of barrier problems, with decreasing barrier parameter. Solver used in this thesis is IPOPT [43]. Article in [39] describes framework that unifies algorithmic components of different solvers, allowing for custom new solvers.

5.1.1 Barrier problem formulation

There are multiple ways to transform NLP in to barrier problem, this section will briefly describe how IPOPT does it. IPOPT article [43] Describes how to transform generic NLP into form 5.1 $h(\mathbf{x}) \geq 0$, interior point methods solve a barrier problem.

$$\begin{aligned} \min_{\mathbf{x}\in\mathbb{R}^n} \quad & \phi_\mu(\mathbf{x}) := f(\mathbf{x}) - \mu \sum_{i=1}^{n} \ln(x_i^{(i)}) \\ \text{s.t.} \quad & c(\mathbf{x}) = 0 \end{aligned} \tag{5.1}$$

Then barrier problem is further transformed into set of linear equations, that compute newton step.

$$\begin{bmatrix} W_k & A_k & -I \\ A_k^T & 0 & 0 \\ Z_k & 0 & X_k \end{bmatrix} \begin{bmatrix} d_x^k \\ d_\lambda^k \\ d_z^k \end{bmatrix} = - \begin{bmatrix} \nabla f(x_k) + A_k\lambda_k - z_k \\ c(x_k) \\ X_k Z_k e - \mu_j e \end{bmatrix} \tag{5.2}$$

There are numerous commercial and free algorithms to solve this. For smaller systems it is LU decomposition, but for larger sparse problems a dedicated solver is a must. IPOPT is most commonly used with MUMPS [7] solver which is free. There are several linear commercial solvers, Harwell Subroutine Library(HSL) supplies ma27 ma87 ma97 [22] pardiso solver [35],

Harwell Subroutine Library solvers are available free for academic usage and pardiso solver is available free for academic but only single core.

5.2 multithreading

linear solver are capable of multithreading, which needs to be explicitly turned on. Tests were performed on Ryzen 9 9950X and MadNLP ran on Nvidia RTX 5070

5.3 Sequential quadratic programming

5.4 Hessian calculation trough Algorithmic differentiation

Mayor complication of many NLP solvers is that hey require Hessian of the objective function in order to compute the step. computation of it can be time intensive. Most universal method is finite difference, that means evaluating the function around current point and estimating the hessian. other method is using symbolic differentiation, often the full symbolic expression is too large to be evaluated effectively due to chain and product rule. Algorithmic differentiation aims to solve problems of previous methods, by appling the chain rule to the elementary operations of a function. Comprehensive review of automatic differentiation is in article [14]

5.5 Scaling and matrix conditioning

Common pitfall of nonlinear programming is poor scaling, this happens when the solver encounters variables, that are orders of magnitude apart, for example motor force in thousands of newton and heading angle in radians. This happens, because at the core of nonlinear programming solver is a linear solver (MUMPS,MA97,PARDISO,...), that has to do matrix factorization every iteration, to solve KKT linear system. If this linear system is ill conditioned due to improper scaling, the factorization may be imprecise and search direction corrupted.

Another thing which led to issues is constraints qualification. Linear indpendence constiant qualification requires gradients of the active constraints to be linearly independent, then the jacobian of the constraints is rank deficient and linear solve may fails

5.6 sparsity

Sparsity is good, sparsity means matrix hessians and jacobians with many zeros also sparsisty in constraints, that means not having super very long

expression with many terms in constraint. I guess it is beacause deriavtive is difficult then and can lead solver in wrong direction or take small steps. Therefore using more decison variables to define same problem can actually lead to faster computation [25]. Certain solvers can exploit the sprasity strcuture, such as FATROP[40]. In source [24] Joris gillis a casadi developer compares IPOPT to FATROP, with **FATROP being 33 times faster on a trajecotry optimisation problem.**

Also [11] describes how sprasity structure helps with afster hessian aprprixmiation for using finite difference??? secant method.

5.7 Initialization

NLP solvers require initial guess for all decision variables, to start solving. For this problem initial guess is obtained driving the vehicle on given track. PD controller is used to keep the car on the centerline and P controller is used to keep velocity at 5m/s. In general, it is possible to initialize all variables at zero, but with large scale nonlinear programming this approach is not used. For this problem initialization with zeros leads to singularity in equation 2.13 and is not possible at all. must be ineitialzied otherwise very baaaadt?

5.8 Global and local optimums

This optimization problem is generally non convex, therefore any local optimum does not guarantee global optimum.Commonly used NLP solvers, like IPOPT guarantee only local optimums and not global ones. Good initization is basis for finding a local optimum close to the global optimum. Source?????

Chapter 6

My implementation, julia + JuMP

I have decided to use julia +JuMP, because I wanted to use free and open source software.

I have already done a lot in Python and julia seemed like new interesting thing to learn. Julia is supposed to be fast and high level language, bridging the gap between C++ and python. Julia packages offer a lot of ready to use mathematical tools developed by community. Julia has

JuMP [27] (Julia for Mathematical Programming) is a modelling language and collection of optimization packages in Julia. JuMP makes it possible to create decision variables and constraints symbolically and uses automatic differentiation, to supply first and second derivative to solvers. Solver used in this thesis is IPOPT [43] and MadNLP[37][36]. MadNLP also allows utilising GPU for solver computation.

6.1 My optimization stack

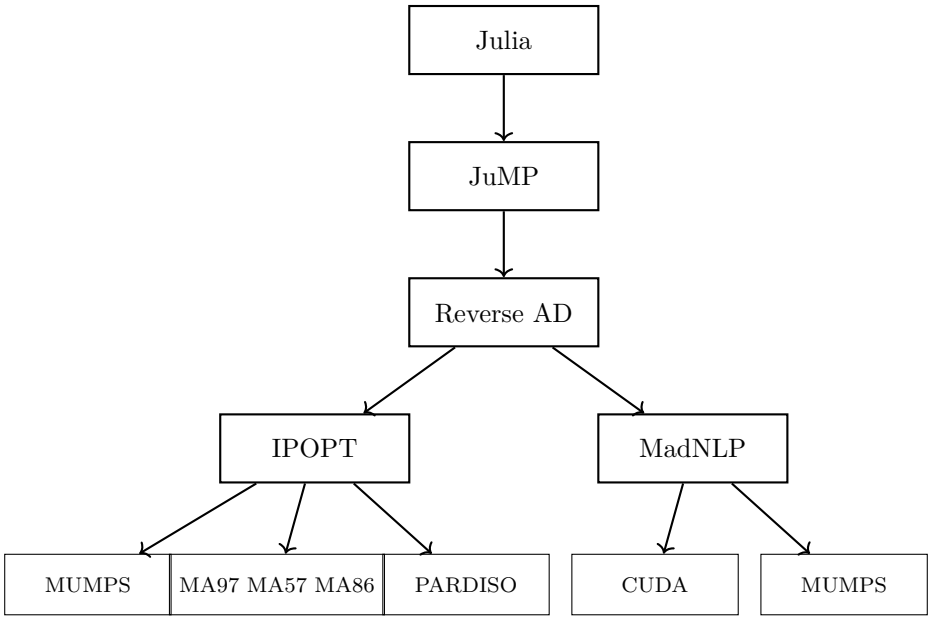


Figure 6.1: Optimization software stack

6.2 Gauss methods as pseudospectral collocation

I have implemented Gauss-Legendre, Gauss-Lobatto, Gauss-Radau as global collocation methods, that is one polynomial approximating the entire trajectory. I have implemented them according to [16]

6.3 Gauss methods as h-method

6.4 Diving problem into many lower degree polynomials

6.5 Mesh refinement methods

6.6 sensitivity Analysis

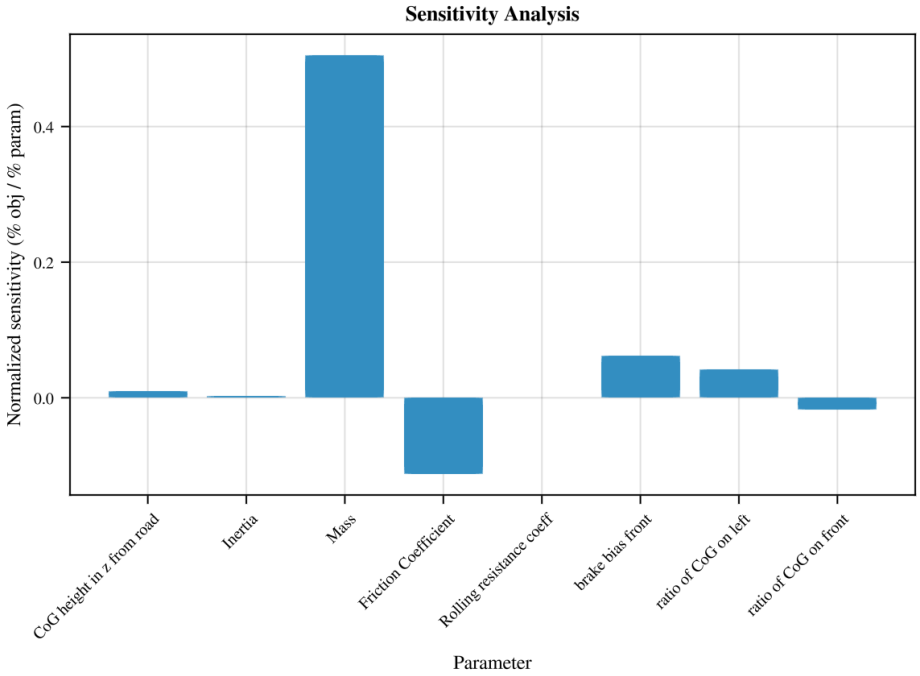


Figure 6.2: Node-interval ODE error for power-test case.

Part III

results

Chapter 7

comparison of methods

Performace of NLP solvers and frameworks is often tested on benchmark problems. like CUTEst [18] dataset?

Table 7.1: Benchmark summary statistics by transcription variant.

var *String*	runs *Int64*	ok *Int64*	fail *Int64*	rate *Float64*	mean_s *Float64*	med_s *Float64*	min_s *Float64*	max_s *Float64*
Legendre	14	10	4	71.4286	50.6255	35.1010	0.6480	134.8663
Lobatto IIIA	14	10	4	71.4286	34.0338	22.1115	0.2849	104.8260
Radau	14	11	3	78.5714	38.5073	26.2869	0.4432	101.0953

Table 7.2: Benchmark results by track, vehicle model, and transcription variant.

track	car	var	ok	status	time_s	obj
String	*String*	*String*	*String*	*String*	*Float64*	*Float64*
FSCZ	singletrack	Radau	yes	solved	28.3320	50.2106
FSCZ	singletrack	Legendre	yes	solved	44.8213	50.5890
FSCZ	singletrack	Lobatto IIIA	yes	solved	23.1096	51.4710
FSCZ	twintrack	Radau	yes	solved	101.0953	57.3381
FSCZ	twintrack	Legendre	no	locally infeasible	378.0604	NaN
FSCZ	twintrack	Lobatto IIIA	no	locally infeasible	222.5674	NaN
Berlin	singletrack	Radau	yes	solved	26.2869	79.4234
Berlin	singletrack	Legendre	yes	solved	45.9145	79.4033
Berlin	singletrack	Lobatto IIIA	yes	solved	39.6746	79.1299
Berlin	twintrack	Radau	yes	solved	68.5323	77.7439
Berlin	twintrack	Legendre	yes	solved	134.8663	77.7883
Berlin	twintrack	Lobatto IIIA	yes	solved	54.0671	77.8983
Berlin	formula	Radau	yes	solved	71.3285	87.9477
Berlin	formula	Legendre	yes	solved	110.9597	88.3235
Berlin	formula	Lobatto IIIA	yes	solved	83.9952	87.9482
Berlin	bus	Radau	no	error	NaN	NaN
Berlin	bus	Legendre	no	error	NaN	NaN
Berlin	bus	Lobatto IIIA	no	error	NaN	NaN
FSG	singletrack	Radau	yes	solved	19.3048	58.2463
FSG	singletrack	Legendre	yes	solved	25.3808	58.1609
FSG	singletrack	Lobatto IIIA	yes	solved	21.1134	56.8773
FSG	twintrack	Radau	yes	solved	89.2777	69.3149
FSG	twintrack	Legendre	yes	solved	117.9348	70.3348
FSG	twintrack	Lobatto IIIA	yes	solved	104.8260	69.8878
DoubleTurn	singletrack	Radau	yes	solved	0.4432	5.7087
DoubleTurn	singletrack	Legendre	yes	solved	0.6480	5.6794
DoubleTurn	singletrack	Lobatto IIIA	yes	solved	0.2849	6.0247
DoubleTurn	twintrack	Radau	yes	solved	2.1308	6.5283
DoubleTurn	twintrack	Legendre	yes	solved	1.3128	6.5835
DoubleTurn	twintrack	Lobatto IIIA	yes	solved	1.6581	7.0205
DoubleTurn	bus	Radau	no	error	NaN	NaN
DoubleTurn	bus	Legendre	no	error	NaN	NaN
DoubleTurn	bus	Lobatto IIIA	no	error	NaN	NaN
FigureEight	singletrack	Radau	yes	solved	2.8865	14.9127
FigureEight	singletrack	Legendre	yes	solved	2.1390	14.9162
FigureEight	singletrack	Lobatto IIIA	yes	solved	1.6345	14.8296
FigureEight	twintrack	Radau	yes	solved	13.9619	16.5690
FigureEight	twintrack	Legendre	yes	solved	22.2782	16.5736
FigureEight	twintrack	Lobatto IIIA	yes	solved	9.9747	16.5319
FigureEight	bus	Radau	no	error	NaN	NaN
FigureEight	bus	Legendre	no	error	NaN	NaN
FigureEight	bus	Lobatto IIIA	no	error	NaN	NaN

Table 7.4: Linear solver benchmark summary statistics.

var	runs	ok	fail	rate	mean_s	med_s	min_s	max_s
String	*Int64*	*Int64*	*Int64*	*Float64*	*Float64*	*Float64*	*Float64*	*Float64*
ma27	14	11	3	78.5714	34.7129	13.0322	0.2789	107.3777
ma57	14	11	3	78.5714	36.2864	12.9839	0.2922	122.8581
ma97	14	11	3	78.5714	36.4128	13.1364	0.2936	114.9782
mumps	14	11	3	78.5714	38.2790	27.4964	0.4200	111.1217

Table 7.6: Linear solver benchmark results by track, vehicle model, and transcription variant.

track	car	var	ok	status	time_s	obj
String	*String*	*String*	*String*	*String*	*Float64*	*Float64*
FSCZ	singletrack	mumps	yes	solved	29.3292	50.2106
FSCZ	singletrack	ma57	yes	solved	10.3380	50.2106
FSCZ	singletrack	ma27	yes	solved	11.1926	50.2106
FSCZ	singletrack	ma97	yes	solved	10.3559	50.2106
FSCZ	twintrack	mumps	yes	solved	111.1217	57.3381
FSCZ	twintrack	ma57	yes	solved	91.2982	57.3381
FSCZ	twintrack	ma27	yes	solved	107.3777	57.2790
FSCZ	twintrack	ma97	yes	solved	92.1819	57.3381
Berlin	singletrack	mumps	yes	solved	27.4964	79.4234
Berlin	singletrack	ma57	yes	solved	24.2503	79.4234
Berlin	singletrack	ma27	yes	solved	25.2657	79.4234
Berlin	singletrack	ma97	yes	solved	25.7588	79.4234
Berlin	twintrack	mumps	yes	solved	66.6356	77.7439
Berlin	twintrack	ma57	yes	solved	58.6398	77.7439
Berlin	twintrack	ma27	yes	solved	50.7242	77.7439
Berlin	twintrack	ma97	yes	solved	55.0071	77.7439
Berlin	formula	mumps	yes	solved	64.9716	87.9477
Berlin	formula	ma57	yes	solved	63.5581	87.9477
Berlin	formula	ma27	yes	solved	59.4152	87.9477
Berlin	formula	ma97	yes	solved	73.5063	87.9477
Berlin	bus	mumps	no	error	NaN	NaN
Berlin	bus	ma57	no	error	NaN	NaN
Berlin	bus	ma27	no	error	NaN	NaN
Berlin	bus	ma97	no	error	NaN	NaN
FSG	singletrack	mumps	yes	solved	19.1169	58.2463
FSG	singletrack	ma57	yes	solved	12.9839	58.2463
FSG	singletrack	ma27	yes	solved	13.0322	58.2463
FSG	singletrack	ma97	yes	solved	13.1364	58.2463
FSG	twintrack	mumps	yes	solved	85.0733	69.3149
FSG	twintrack	ma57	yes	solved	122.8581	69.3471
FSG	twintrack	ma27	yes	solved	100.1829	69.3140
FSG	twintrack	ma97	yes	solved	114.9782	69.3140
DoubleTurn	singletrack	mumps	yes	solved	0.4200	5.7087
DoubleTurn	singletrack	ma57	yes	solved	0.2922	5.7087
DoubleTurn	singletrack	ma27	yes	solved	0.2789	5.7087
DoubleTurn	singletrack	ma97	yes	solved	0.2936	5.7087
DoubleTurn	twintrack	mumps	yes	solved	2.0388	6.5283
DoubleTurn	twintrack	ma57	yes	solved	2.8234	6.5283
DoubleTurn	twintrack	ma27	yes	solved	1.8918	6.5283
DoubleTurn	twintrack	ma97	yes	solved	1.8591	6.5283
DoubleTurn	bus	mumps	no	error	NaN	NaN
DoubleTurn	bus	ma57	no	error	NaN	NaN
DoubleTurn	bus	ma27	no	error	NaN	NaN
DoubleTurn	bus	ma97	no	error	NaN	NaN
FigureEight	singletrack	mumps	yes	solved	1.7804	14.9127
FigureEight	singletrack	ma57	yes	solved	0.9935	14.9127
FigureEight	singletrack	ma27	yes	solved	1.0276	14.9127
FigureEight	singletrack	ma97	yes	solved	1.7419	14.9127
FigureEight	twintrack	mumps	yes	solved	13.0852	16.5690
FigureEight	twintrack	ma57	yes	solved	11.1149	16.5690
FigureEight	twintrack	ma27	yes	solved	11.4525	16.5690
FigureEight	twintrack	ma97	yes	solved	11.7215	16.5690
FigureEight	bus	mumps	no	error	NaN	NaN
FigureEight	bus	ma57	no	error	NaN	NaN
FigureEight	bus	ma27	no	error	NaN	NaN
FigureEight	bus	ma97	no	error	NaN	NaN

Table 7.8: NLP solver / backend benchmark summary statistics (IPOPT-MUMPS, MadNLP-CPU, MadNLP-GPU).

var	runs	ok	fail	rate	mean_s	med_s	min_s	max_s
String	*Int64*	*Int64*	*Int64*	*Float64*	*Float64*	*Float64*	*Float64*	*Float64*
ipopt-mumps	14	11	3	78.5714	37.8884	17.5527	0.4463	97.7375
madnlp-cpu	14	11	3	78.5714	35.9528	19.0218	0.4192	107.1928
madnlp-gpu	14	11	3	78.5714	35.9844	20.9505	0.5009	96.0756

Table 7.10: NLP solver / backend benchmark results by track, vehicle model, and transcription variant.

track	car	var	ok	status	time_s	obj
String	*String*	*String*	*String*	*String*	*Float64*	*Float64*
FSCZ	singletrack	ipopt-mumps	yes	solved	13.1230	50.2106
FSCZ	singletrack	madnlp-cpu	yes	solved	12.0456	50.2106
FSCZ	singletrack	madnlp-gpu	yes	solved	11.4359	50.2098
FSCZ	twintrack	ipopt-mumps	yes	solved	97.7375	57.3381
FSCZ	twintrack	madnlp-cpu	yes	solved	107.1928	57.2490
FSCZ	twintrack	madnlp-gpu	yes	solved	86.5849	57.2521
Berlin	singletrack	ipopt-mumps	yes	solved	27.8981	79.4234
Berlin	singletrack	madnlp-cpu	yes	solved	19.0218	79.4234
Berlin	singletrack	madnlp-gpu	yes	solved	26.0687	79.4214
Berlin	twintrack	ipopt-mumps	yes	solved	89.0314	77.7417
Berlin	twintrack	madnlp-cpu	yes	solved	61.1344	77.7442
Berlin	twintrack	madnlp-gpu	yes	solved	47.4224	77.7421
Berlin	formula	ipopt-mumps	yes	solved	66.6727	87.9477
Berlin	formula	madnlp-cpu	yes	solved	61.3201	87.9477
Berlin	formula	madnlp-gpu	yes	almost locally solved	89.3990	87.9190
Berlin	bus	ipopt-mumps	no	error	NaN	NaN
Berlin	bus	madnlp-cpu	no	error	NaN	NaN
Berlin	bus	madnlp-gpu	no	error	NaN	NaN
FSG	singletrack	ipopt-mumps	yes	solved	17.5527	58.2463
FSG	singletrack	madnlp-cpu	yes	solved	12.8914	58.2463
FSG	singletrack	madnlp-gpu	yes	solved	13.2424	58.2455
FSG	twintrack	ipopt-mumps	yes	solved	87.8248	69.3149
FSG	twintrack	madnlp-cpu	yes	solved	98.1893	69.4039
FSG	twintrack	madnlp-gpu	yes	solved	96.0756	69.3784
DoubleTurn	singletrack	ipopt-mumps	yes	solved	0.4463	5.7087
DoubleTurn	singletrack	madnlp-cpu	yes	solved	0.4192	5.7087
DoubleTurn	singletrack	madnlp-gpu	yes	solved	0.5009	5.7086
DoubleTurn	twintrack	ipopt-mumps	yes	solved	2.1846	6.5283
DoubleTurn	twintrack	madnlp-cpu	yes	solved	2.8648	6.5463
DoubleTurn	twintrack	madnlp-gpu	yes	solved	2.7804	6.5282
DoubleTurn	bus	ipopt-mumps	no	error	NaN	NaN
DoubleTurn	bus	madnlp-cpu	no	error	NaN	NaN
DoubleTurn	bus	madnlp-gpu	no	error	NaN	NaN
FigureEight	singletrack	ipopt-mumps	yes	solved	1.7208	14.9127
FigureEight	singletrack	madnlp-cpu	yes	solved	0.9006	14.9127
FigureEight	singletrack	madnlp-gpu	yes	solved	1.3678	14.9124
FigureEight	twintrack	ipopt-mumps	yes	solved	12.5807	16.5690
FigureEight	twintrack	madnlp-cpu	yes	solved	19.5003	16.5658
FigureEight	twintrack	madnlp-gpu	yes	solved	20.9505	16.5656
FigureEight	bus	ipopt-mumps	no	error	NaN	NaN
FigureEight	bus	madnlp-cpu	no	error	NaN	NaN
FigureEight	bus	madnlp-gpu	no	error	NaN	NaN

Chapter 8

Conclusion

Appendices

Appendix A

Bibliography

[1] 2026 Class Project: Race Car Minimum Lap Time - AE6310 Course Notes. https://smdogroup.github.io/ae6310/jupyter/2026_project/race_car_problem.html.

[2] A Computationally Efficient Framework for Free-trajectory Minimum-lap-time Optimization of Racing Cars. https://arxiv.org/html/2511.13522v1.

[3] OptimumLap | OptimumG.

[4] Solving ordinary differential equations I. nonstiff problems. *Mathematics and Computers in Simulation*, 29(5):447, October 1987.

[5] Coin-or/CppAD. COIN-OR Foundation, May 2026.

[6] Yunus Agamawi and Anil Rao. CGPOPS: A C++ Software for Solving Multiple-Phase Optimal Control Problems Using Adaptive Gaussian Quadrature Collocation and Sparse Nonlinear Programming. *ACM Transactions on Mathematical Software*, 46, May 2020.

[7] P.R. Amestoy, I. S. Duff, J. Koster, and J.-Y. L'Excellent. A fully asynchronous multifrontal solver using distributed dynamic scheduling. *SIAM Journal on Matrix Analysis and Applications*, 23(1):15–41, 2001.

[8] Uri M. Ascher, Robert M. M. Mattheij, and Robert D. Russell. *Numerical Solution of Boundary Value Problems for Ordinary Differential Equations*. Classics in Applied Mathematics. SIAM, 1995.

[9] Jean-Paul Berrut and Lloyd N. Trefethen. Barycentric Lagrange Interpolation. *SIAM Review*, 46(3):501–517, January 2004.

[10] Mathieu Besançon, Joaquim Dias Garcia, Benoît Legat, and Akshay Sharma. Flexible differentiable optimization via model transformations. *INFORMS Journal on Computing*, 36(2):456–478, 2023.

[11] John T. Betts. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*. Society for Industrial and Applied Mathematics, second edition, January 2010.

[12] Forbes T. Brown. *Engineering System Dynamics: A Unified Graph-Centered Approach, Second Edition*. CRC Press, Boca Raton, 2 edition, August 2006.

[13] Richard Ciglanský. Analysis of vehicle components and their impact on lap-time.

[14] Daniel Cortild, J Haastert, A Sanabria, and F Voronine. *A Brief Review of Automatic Differentiation*. March 2023.

[15] Christopher L. Darby, William W. Hager, and Anil V. Rao. An *hp* -adaptive pseudospectral method for solving optimal control problems. *Optimal Control Applications and Methods*, 32(4):476–502, July 2011.

[16] Divya Garg, Michael Patterson, William Hager, Anil Rao, David R Benson, and Geoffrey T Huntington. An overview of three pseudospectral methods for the numerical solution of optimal control problems.

[17] Divya Garg, Michael Patterson, William W. Hager, Anil V. Rao, David A. Benson, and Geoffrey T. Huntington. A unified framework for the numerical solution of optimal control problems using pseudospectral methods. *Automatica*, 46(11):1843–1851, November 2010.

[18] Nick Gould, Dominique Orban, and Philippe Toint. CUTEst: A Constrained and Unconstrained Testing Environment with safe threads for Mathematical Optimization. *Computational Optimization and Applications*, 60, July 2014.

[19] A. Harten, B. Engquist, S. Osher, and S. R. Chakravarthy. Uniformly high order accurate essentially non-oscillatory schemes, III. *Journal of Computational Physics*, 131(1):3–47, 1997.

[20] Himanshu Hatwal and E. C. Mikulcik. Some inverse solutions to an automobile path-tracking problem with input control of steering and brakes. *Vehicle System Dynamics*, 15:61–71, 1986.

[21] Alexander Heilmeier, Maximilian Geisslinger, and Johannes Betz. A quasi-steady-state lap time simulation for electrified race cars. In *2019 Fourteenth International Conference on Ecological Vehicles and Renewable Energies (EVER)*. IEEE, May 2019.

[22] HSL. A collection of Fortran codes for large scale scientific computation. `http://www.hsl.rl.ac.uk/`, 2011.

[23] Shuang Li∗ Jisong Zhao. Adaptive mesh refinement method for solving optimal control problems using interpolation error analysis and improved data compression.

[24] Joris Gillis. IPOPT on steroids: Fatrop solver in CasADi, June 2024.

[25] Julia Discourse. 90% of IPOPT run time is spent in Hessian-constraints evaluation, very little time in linear solver — is it normal? https://discourse.julialang.org/t/90-of-ipopt-run-time-is-spent-in-hessian-constraints-evaluation-very-little-ti 136708, 2025. Accessed: 2026-04-19.

[26] Fengjin Liu, William W. Hager, and Anil V. Rao. Adaptive Mesh Refinement Method for Optimal Control Using Decay Rates of Legendre Polynomial Coefficients. *IEEE Transactions on Control Systems Technology*, 26(4):1475–1483, July 2018.

[27] Miles Lubin, Oscar Dowson, Joaquim Dias Garcia, Joey Huchette, Benoît Legat, and Juan Pablo Vielma. JuMP 1.0: Recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*, 2023.

[28] Juan Manzanero. Juanmanzanero/fastest-lap, April 2026.

[29] Matteo Massaro and David Limebeer. Minimum-lap-time optimisation and simulation. *Vehicle System Dynamics*, 59:1–45, April 2021.

[30] mc12027. Mc12027/OpenLAP-Lap-Time-Simulator, April 2026.

[31] Yukio Nakajima. Advanced tire mechanics. 2019.

[32] Giacomo Perantoni and David J.N. Limebeer. Optimal control for a Formula One car with variable parameters. *Vehicle System Dynamics*, 52(5):653–678, May 2014.

[33] Anil Rao. A Survey of Numerical Methods for Optimal Control. *Advances in the Astronautical Sciences*, 135, January 2010.

[34] R. D. Roland and C. F. Thelin. Computer simulation of Watkins Glen grand prix circuit performance. Technical Report ZL-5002-K-1, Cornell Aeronautical Laboratory, 1971.

[35] Olaf Schenk, Klaus Gärtner, Wolfgang Fichtner, and A.D. Stricker. Pardiso: A high-performance serial and parallel sparse linear solver in semiconductor device simulation. *Future Generation Computer Systems*, 18:69–78, 03 2000.

[36] Sungho Shin, Mihai Anitescu, and François Pacaud. Accelerating optimal power flow with GPUs: SIMD abstraction of nonlinear programs and condensed-space interior-point methods. *Electric Power Systems Research*, 236:110651, 2024.

[37] Sungho Shin, Carleton Coffrin, Kaarthik Sundar, and Victor M Zavala. Graph-based modeling and decomposition of energy infrastructures. *IFAC-PapersOnLine*, 54(3):693–698, 2021.

[38] Sungho Shin, François Pacaud, and Mihai Anitescu. Accelerating optimal power flow with GPUs: SIMD abstraction of nonlinear programs and condensed-space interior-point methods, 2023.

[39] Charlie Vanaret and Sven Leyffer. *Implementing a unified solver for nonlinearly constrained optimization*. November 2025.

[40] Lander Vanroye, Ajay Sathya, Joris De Schutter, and Wilm Decré. FATROP : A Fast Constrained Optimal Control Problem Solver for Robot Trajectory Optimization and Control, July 2023.

[41] Matthew E. Wilhelm and Matthew D. Stuber. EAGO.jl: easy advanced global optimization in Julia. *Optimization Methods and Software*, 37(2):425–450, 2022.

[42] Hendrik Wilka and Jens Lang. Adaptive hp-Polynomial Based Sparse Grid Collocation Algorithms for Piecewise Smooth Functions with Kinks, May 2025.

[43] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, March 2006.